

数据结构课程设计

接口文档

2025 年 5 月

目录

1 Attractions 模块	1
1.1 获取景点列表接口	1
1.2 景点点赞接口	2
1.3 景点取消点赞接口	3
1.4 获取用户已点赞景点列表接口	4
1.5 获取基于用户协同过滤的景点推荐	5
2 Routing 模块	6
2.1 获取图数据接口	6
2.2 使用 Dijkstra 算法计算最短路径接口	9
2.3 计算经过指定节点的最短回路接口	10
2.4 雷达搜索接口（场所查询接口）	10
3 Journals 模块	12
3.1 获取日记列表接口	12
3.2 日记点赞接口	13
3.3 日记取消点赞接口	14
3.4 获取日志详情接口	14
3.5 获取用户已点赞日记列表接口	15
3.6 获取景点 ID 和名称列表接口	17
3.7 创建日记接口	17
4 图片数据访问接口	19
5 登录与注册接口	19
5.1 用户登录接口	19
5.2 用户注册接口	20

1 Attractions 模块

1.1 获取景点列表接口

接口地址：/attractions/list_attractions/

请求方法：GET

请求参数：

参数名	类型	是否必传	描述
page_num	int	否	当前页码，默认为 0
page_size	int	否	每页数量，默认为 30
keyword	str	否	关键词筛选，用于搜索景点名称和描述 (空格隔开的关键词)
type	str	否	景点类型筛选 ('校园', '景区', 不传)
sort_type	str	否	排序类型 ('comments', 'likes', 不传)
sort_direction	str	否	排序方向 ('rev', 不传)
user_id	int	否	当前用户的 ID

响应数据：

成功：

```
{
  "ret": 0,
  "data": {
    "length": <int>, # 当前页数据数量
    "retlist": <list>, # 景点列表
    "current_page": <int>, # 当前页码
    "page_size": <int>, # 每页数量
    "total_pages": <int> # 总页数
  }
}
```

其中 retlist 中的元素形如：

```
"retlist": [
{
  "id": 1,
  "cnName": "故宫博物院",
  "enName": "The Palace Museum",
  "img_url": "https://img1.qunarzz.com/travel/d9/1803/c1/46a6398c92f17fb5.jpg_r_480x360x95_d05b4420.jpg",
  "description": "世界五大宫之首，是中国明清两代的皇家宫殿。",
  "num_comment_quna": 31325,
  "num_likes": 4,
  "type": "景区",
  "num_journals": 8,
  "is_liked": false
},

```

其中 img_url 是一个完整的图片链接，不需要经过代理接口。

失败:

```
{
  "ret": 1,
  "message": "<错误信息>"
}
```

1.2 景点点赞接口

接口地址: /attractions/like_attractions/

请求方法: POST

请求体:

```
{
  "user_id": <int>, # 用户ID
  "post_id": <int> # 帖子ID
}
```

响应数据:

成功:

```
{
  "status": "success",
  "action": "Liked", # 表示点赞成功
  "likes_count": <int> # 当前帖子的总点赞数
}
```

如果已经点赞但是又请求点赞（正常情况下前端不允许这个操作），'action'会变成'already_liked'，仍然认为是一个成功返回。

失败:

```
{  
  "error": "<错误信息>"  
}
```

1.3 景点取消点赞接口

接口地址: /attractions/unlike_attractions/

请求方法: POST

请求体:

```
{  
  "user_id": <int>,  
  "post_id": <int>  
}
```

响应数据:

成功:

```
{  
  "status": "success",  
  "action": "unLiked",  
  "likes_count": <int> # 取消点赞后的点赞数  
}
```

如果没有点赞但是又请求取消点赞(正常情况下前端不允许这个操作), 'action' 会变成 'not_liked', 仍然认为是一个成功返回。

失败:

```
{  
  "error": "<错误信息>"  
}
```

1.4 获取用户已点赞景点列表接口

接口地址：/attractions/list_attractions_user/

请求方法：GET

请求参数：

参数名	类型	是否必传	描述
page_num	int	否	当前页码，默认为 0
page_size	int	否	每页数量，默认为 30
user_id	int	是	当前用户的 ID

响应数据：（和接口 1.1 一致）

成功：

```
{
  "ret": 0,
  "data": {
    "length": <int>, # 当前页数据数量
    "retlist": <List>, # 景点列表
    "current_page": <int>, # 当前页码
    "page_size": <int>, # 每页数量
    "total_pages": <int> # 总页数
  }
}
```

其中 retlist 中的元素形如：

```
"retlist": [
  {
    "id": 1,
    "cnName": "故宫博物院",
    "enName": "The Palace Museum",
    "img_url": "https://img1.qunarzz.com/travel/d9/1803/c1/46a6398c92f17fb5.jpg_r_480x360x95_d05b4420.jpg",
    "description": "世界五大宫之首，是中国明清两代的皇家宫殿。",
    "num_comment_quna": 31325,
    "num_likes": 4,
    "type": "景区",
    "num_journals": 8,
    "is_liked": false
  },
  ...
]
```

失败：

```
{
  "ret": 1,
  "message": "<错误信息>"
}
```

1.5 获取基于用户协同过滤的景点推荐

接口地址：/attractions/list_UBCF/

请求方法：GET

请求参数：

参数名	类型	是否必传	描述
user_id	int	是	当前用户的 ID
count	Int	否	需要的推荐数量，默认为 10

响应数据：（和接口 1.1 一致）

成功：

```
成功:
{
  "ret": 0,
  "data": {
    "recommendations": <List> # 推荐的景点列表
  }
}
```

其中 recommendations 中的元素形如：

```
{
  "id": 6,
  "cnName": "天安门广场",
  "enName": "Tiananmen Square",
  "img_url": "https://imgl.qunarzz.com/travel/d6/1611/4a/1e5ed390b30ed1b5.jpg_r_480x360x95_556e77c2.jpg",
  "description": "伟大祖国的标志，早起看一场庄严肃穆的升旗仪式，瞻仰人民英雄纪念碑。",
  "num_comment_quna": 7509,
  "num_likes": 4,
  "type": "景区",
  "num_journals": 5,
  "recommendation_score": 1.25910364145658,
  "is_liked": false
},
```

这个元素只比之前 retlist 中的元素多了一个 recommendation_score 推荐度字段。

失败：

```
{
  "ret": 1,
  "message": "<错误信息>"
}
```

2 Routing 模块

2.1 获取图数据接口

接口地址: /routing/list_graph/

请求方法: GET

请求参数:

参数名	类型	是否必传	描述
id	int	是	图的 ID, 默认为 0

响应数据:


```
{
  "ret": 0,
  "data": {
    "nodes": [
      {
        "id": <int>,
        "name": <str>,
        "type": <str>,
        "x": <float>,
        "y": <float>,
        "details": {
          "description": <str>,
          "image": <str>
        }
      }
    ],
    "edges": [
      {
        "from": <int>,
        "to": <int>,
        "vehicle": <str>
      }
    ],
    "center": {
      "x": <float>,
      "y": <float>
    },
    "name": <str>,
    "types": {
      "spotTypes": {
        "办公楼": {
          "name": <str>,
          "color": <str>,
          "icon": <str>
        },
        // 其他景点类型...
      },
      "pathTypes": {
        "walk": {
          "color": <str>,
          "weight": <int>
        },
        // 其他路径类型...
      }
    },
    "bounds": {
      "virtual": {
        "minX": <int>,
        "maxX": <int>,
        "minY": <int>,
        "maxY": <int>
      },
      "real": {
        "minLng": <float>,
        "maxLng": <float>,
        "minLat": <float>,
        "maxLat": <float>
      }
    }
  }
}
```

其中, "types" 参数的第一个参数 "spotTypes" 是:

```
spot_types = {
  "办公楼": {
    "name": "办公楼",
    "color": "#3F51B5",
    "icon": "🏢"
  },
  "地标": {
    "name": "地标",
    "color": "#FFC107",
    "icon": "📍"
  },
  "宿舍楼": {
    "name": "宿舍楼",
    "color": "#4CAF50",
    "icon": "🏠"
  },
  "教学楼": {
    "name": "教学楼",
    "color": "#00BCD4",
    "icon": "🎓"
  },
  "体育馆": {
    "name": "体育馆",
    "color": "#F44336",
    "icon": "🏟️"
  },
  "厕所": {
    "name": "厕所",
    "color": "#9E9E9E",
    "icon": "🚻"
  },
  "咖啡": {
    "name": "咖啡",
    "color": "#795548",
    "icon": "☕"
  },
  "商店": {
    "name": "商店",
    "color": "#673AB7",
    "icon": "🛒"
  },
  "图书馆": {
    "name": "图书馆",
    "color": "#03A9F4",
    "icon": "📖"
  },
  "操场": {
    "name": "操场",
    "color": "#8BC34A",
    "icon": "🏟️"
  },
  "活动中心": {
    "name": "活动中心",
    "color": "#FF9800",
    "icon": "🎡"
  },
  "超市": {
    "name": "超市",
    "color": "#388E3C",
    "icon": "🛒"
  },
  "食堂": {
    "name": "食堂",
    "color": "#FF5722",
    "icon": "🍴"
  },
  "饭店": {
    "name": "饭店",
    "color": "#FF6347",
    "icon": "🍴"
  },
  "路口": {
    "name": "路口",
    "color": "#FFEB3B",
    "icon": "🚦"
  }
}
```

这个参数规定了节点的显示样式, 长 15, 这 15 个 keys 值, 就是 data 参数中 nodes 参数中每个节点数据的 type 参数所有可能的取值。

其中, "types" 参数的第二个参数 "path_types" 是:

```
path_types = {
  'walk': {
    'color': '#696969',
    'weight': 5,
  },
  'bike': {
    'color': '#A020F0',
    'weight': 5,
  },
  'both': {
    'color': '#4B0082',
    'weight': 10,
  },
}
```

这个参数规定了边的显示样式, 长 3, 这 3 个 keys 值 (walk, bike, both), 就是 data 参数中 edges 参数中每个边数据的 vehicle 参数所有可能的取值。

其中, 'bounds'参数帮助前端确定节点映射在地图上的东西南北范围, 'center'锚定了进入地图镜头的位置。

2.2 使用 Dijkstra 算法计算最短路径接口

接口地址: /routing/best_path_dijkstra/

请求方法: GET

请求参数:

参数名	类型	是否必传	描述
weight	str	是	权重类型, 取值为 'distance', 'time', 'walk_time'
start	int	是	起点节点 ID
end	int	是	终点节点 ID
id	int	是	图的 ID

相应数据:

```
{
  "ret": 0,
  "data": {
    "dist": <int or float>,
    "path": <list>
  }
}
```

这个接口, 还有以下的两个接口, 都含有 weight 参数。取值'distance'意味着最短路程, 取值'time'意味着既可以走车也可以走路的最短时间, 取值'walk_time'意味着只可以走路的最短时间。

2.3 计算经过指定节点的最短回路接口

接口地址: /routing/best_path_circuit/

请求方法: GET

请求参数:

参数名	类型	是否必传	描述
weight	str	是	权重类型, 取值为 'distance', 'time', 'walk_time'
start	int	是	起点节点 ID
nodes	str	是	需经过的节点 ID 列表的 JSON 字符串, 如 "[1, 2, 3]"
id	int	是	图的 ID

响应数据:

```
{
  "ret": 0,
  "data": {
    "dist": <int or float>,
    "path": <list>,
    "path_extracted": <list>
  }
}
```

其中 path 参数表示整个路径上要经过的节点, 是前端用到的参数,

path_extracted 是一个未用到的参数, 意味着这些必经点 (包括起点) 在这个路径中的经过顺序。未来可以考虑强调这个数据。

2.4 雷达搜索接口 (场所查询接口)

接口地址: /routing/radar_search/

请求方法：GET

请求参数：

参数名	类型	是否必传	描述
weight	str	是	权重类型，取值为 'distance', 'time', 'walk_time'
start	int	是	起点节点 ID
k	int	是	搜索的节点数量
node_type	str	否	节点类型，默认为 None
id	int	是	图的 ID

响应数据：

```
result = {
    'ret': 0,
    'data': g.ra
}
```

其中，data 是从近到远的 k 个场所的 id 值。

3 Journals 模块

3.1 获取日记列表接口

接口地址: /journals/list_journals/

请求方法: GET

请求参数:

参数名	类型	是否必传	描述
page_num	int	否	当前页码, 默认为 0
page_size	int	否	每页数量, 默认为 30
keyword	str	否	关键词筛选, 用于搜索日记标题和内容 (空格隔开的关键词)
spot_id	str	否	景点 ID 筛选参数 (为了稳定, 使用 str)
uploader_id	str	否	上传者 ID 筛选参数 (为了稳定, 使用 str)
sort_type	Str	否	排序类型 ('likes', 'date')
sort_direction	str	否	排序方向 ('asc' 升序, 'desc' 降序)
user_id	int	否	当前用户的 ID

响应数据:

成功:

```
{
  "ret": 0,
  "data": {
    "length": <int>, # 当前页数据数量
    "retlist": <list>, # 日记列表, 每个日记包含以下信息:
      - 'spot_name': 景点名称
      - 'img_cover': 日记的封面图片 (取第一张图片)
      - 'user_name': 上传者的用户名
      - 'is_liked': 当前用户是否点赞该日记
    "current_page": <int>, # 当前页码
    "page_size": <int>, # 每页数量
    "total_pages": <int> # 总页数
  }
}
```

其中 retlist 中的元素形如：

```
"retlist": [
{
  "journal_id": 1,
  "title": "故宫一日",
  "date": "2022-08-15",
  "spot_id": 1,
  "imgs": [
    "img0.jpg",
    "img1.jpg"
  ],
  "content": "故宫太震撼了！红墙金瓦特别壮观，走一圈就感受到古代皇家的威严。就是旺季人太多，建议错峰来！",
  "user_id": 1,
  "num_likes": 3,
  "spot_name": "故宫博物院",
  "img_cover": "img0.jpg",
  "user_name": "111111",
  "is_liked": false
},

```

失败：

```
{
  "ret": 1,
  "message": "<错误信息>"
}
```

3.2 日记点赞接口

接口地址：/journals/like_journals/

请求方法：POST

请求体：

```
{
  "user_id": <int>,
  "post_id": <int>
}
```

响应数据：

成功：

```
{
  "status": "success",
  "action": "Liked", # 表示点赞成功
  "likes_count": <int> # 当前帖子的总点赞数
}
```

如果已经点赞但是又请求点赞（正常情况下前端不允许这个操作），'action'会变成'already_liked'，仍然认为是一个成功返回。

失败：

```
{
  "error": "<错误信息>"
}
```

3.3 日记取消点赞接口

接口地址：/journals/unlike_journals/

请求方法：POST

请求体：

```
{
  "user_id": <int>,
  "post_id": <int>
}
```

响应数据：

成功：

```
{
  "status": "success",
  "action": "unliked",
  "likes_count": <int> # 取消点赞后的点赞数
}
```

失败：

```
{
  "error": "<错误信息>"
}
```

3.4 获取日志详情接口

接口地址：/journals/list_journals_detail/

请求方法：GET

请求参数：

参数名	类型	是否必传	描述
journal_id	int	是	日志 ID
user_id	int	否	用户 ID，用于判断是否点赞

响应数据：

```
{
  "ret": 0, # 状态码, 0表示成功, 非0表示失败
  "journal": { # 日志详情
    "journal_id": int, # 日志ID
    "title": str, # 日志标题
    "date": str, # 日期, 格式YYYY-MM-DD
    "spot_id": int, # 景点ID
    "spot_name": str, # 景点名称 (通过spot_id转换)
    "imgs": list[str], # 图片列表
    "img_cover": str or None, # 封面图片 (取imgs第一张), 无图片时为None
    "content": str, # 日志内容
    "user_id": int, # 用户ID
    "user_name": str, # 用户名
    "num_likes": int, # 点赞总数
    "is_liked": bool # 当前用户是否已点赞
  }
}
```

注：这是一个只用在详情页面的接口。

3.5 获取用户已点赞日记列表接口

接口地址：/journals/list_journals_user/

请求方法：GET

请求参数：

参数名	类型	是否必传	描述
page_num	int	否	当前页码，默认为 0

page_size	int	否	每页数量，默认为 30
user_id	int	是	当前用户的 ID

响应数据：（和接口 3.1 一致）

成功：

```
{
  "ret": 0,
  "data": {
    "length": <int>, # 当前页数据数量
    "retlist": <list>, # 日记列表，每个日记包含以下信息：
      - 'spot_name': 景点名称
      - 'img_cover': 日记的封面图片（取第一张图片）
      - 'user_name': 上传者的用户名
      - 'is_liked': 当前用户是否点赞该日记
    "current_page": <int>, # 当前页码
    "page_size": <int>, # 每页数量
    "total_pages": <int> # 总页数
  }
}
```

其中 retlist 中的元素形如：

```
"retlist": [
  {
    "journal_id": 1,
    "title": "故宫一日",
    "date": "2022-08-15",
    "spot_id": 1,
    "imgs": [
      "img0.jpg",
      "img1.jpg"
    ],
    "content": "故宫太震撼了！红墙金瓦特别壮观，走一圈就感受到古代皇家的威严。就是旺季人太多，建议错峰来！",
    "user_id": 1,
    "num_likes": 3,
    "spot_name": "故宫博物院",
    "img_cover": "img0.jpg",
    "user_name": "111111",
    "is_liked": false
  },
]
```

失败：

```
{
  "ret": 1,
  "message": "<错误信息>"
}
```

3.6 获取景点 ID 和名称列表接口

接口地址: /journals/list_attraction_id_and_name/

请求方法: GET

请求参数: 无

响应数据:

```
{
  "ret": 0,
  "data": <list of tuples>, // 包含景点ID和名称的元组列表 [(id1, name1), (id2, name2), ...]
}
```

注: 这是一个方便发布日记页面方便展示所用的到的接口, 为了减轻发布页面初始创建时请求的数据量。

3.7 创建日记接口

接口地址: /journals/create_journal/

请求方法: POST

请求体:

参数名	类型	是否必传	描述
title	str	是	日记标题
date	str	是	日期, 格式 YYYY-MM-DD
spot_id	int	是	景点 ID
content	str	是	日记内容
user_id	int	是	用户 ID
imgs	list of files	否	图片文件列表

响应数据:

成功:

```
{
  "ret": 0,
  "data": {
    "title": <str>,
    "date": <str>,
    "spot_id": <int>,
    "content": <str>,
    "user_id": <int>,
    "imgs": <list[str]> // 处理后的文件名列表
  }
}
```

失败:

```
{
  "ret": 1,
  "msg": "<错误信息>"
}
```

4 图片数据访问接口

前端只有 Journals 模块的图片数据需要经过代理来访问服务器的图片，因为这一模块接口只提供了文件名。其他图片链接接收下来直接请求互联网即可。

接口地址：/journals/get_journal_image/{文件名}

请求方法：GET

返回值直接是图片格式的数据流。

5 登录与注册接口

5.1 用户登录接口

接口地址：/users/login/

请求方法：POST

请求体：

```
{
  "username": "<str>", // 用户名
  "password": "<str>" // 密码
}
```

响应数据：

成功：

```
{
  "token": "<str>", // JWT访问令牌
  "user": {
    "id": <int>, // 用户ID
    "username": "<str>" // 用户名
  }
}
```

失败:

```
{
  "error": "用户名或密码错误"
}
```

5.2 用户注册接口

接口地址: /users/register/

请求方法: POST

请求体:

```
{
  "username": "<str>", // 用户名
  "password": "<str>" // 密码
}
```

响应数据:

成功:

```
{
  "token": "<str>", // JWT访问令牌
  "user": {
    "id": <int>, // 用户ID
    "username": "<str>" // 用户名
  }
}
```

失败:

```
{
  "error": "该用户名已被注册"
}
```